

Robot Remote Control Working Group

[YOUR FULL NAME]

Request for Comments: RRCF-0001

Indo-Mars Technologies

Category: Standards Track

Version 0.2 DRAFT July 8, 2026

RRCF: Robot Remote Control Format

Operator Interface Declaration Standard for Physical AI

Version 1.0 Draft — Apache License 2.0

We are entering an era of robot proliferation without historical precedent. From sub-millimeter medical nanorobots to multi-ton industrial humanoids, from agricultural swarms covering thousands of acres to personal service robots in every home — the diversity and scale of robotic deployment will reach billions of units within this decade.

In every deployment context, a human in the loop is inevitable. A surgeon monitors a medical robot. A farmer supervises an agricultural swarm. A warehouse manager oversees a mixed fleet. A first responder needs to halt a malfunctioning robot. A child interacts with a home assistant. An AI system generates commands for a physical agent. At every level — from direct teleoperation to high-level supervision to emergency intervention — humans will interact with robots constantly, every day, at all times.

Today, every robot requires its own controller, its own interface, its own training. This fragmentation is unsustainable at scale. Just as the proliferation of automobiles required standardized controls so that any licensed driver could operate any vehicle safely, the proliferation of billions of robots demands a global common operator interface standard. Without it, robot operation remains a specialist skill, safety is inconsistent, fleet management is fragmented, and the potential of Physical AI is bottlenecked by interface incompatibility.

Billions of robots are coming — from nano to humanoid. Every human will deal with robots every day. A global common operator interface standard is not optional. RRCF is that standard.

Abstract

We present RRCF (Robot Remote Control Format), the first Operator Interface Declaration Standard for robotic platforms. RRCF defines a machine-readable file format (.rrcf) in which robot manufacturers declare their complete operator control interface — morphology category, locomotion modes, skill repertoire, HUD display fields, joystick axis semantics, input modalities (touch, HID gamepad, voice, gesture, XR), attachment control panels, custom operator controls, safety limits, and transport endpoints. Any RRCF-compliant controller — web UI, fleet dashboard, or VLA model — reads this declaration and renders the correct operator interface without per-robot code. Human operators use the rendered web UI; AI systems load the .rrcf profile directly at session start via retrieval-augmented generation (RAG for robots). RRCF complements URDF, VDA 5050, Open-RMF, and NVIDIA Halos as the missing operator interface layer. Published Apache 2.0.

1. Motivation — The Human-Robot Interface Imperative

We are entering an era of robot proliferation without historical precedent. From sub-millimeter medical nanorobots to multi-ton industrial humanoids, from agricultural swarms covering thousands of acres to personal service robots in every home — the diversity and scale of robotic deployment will reach billions of units within this decade.

In every deployment context, a human in the loop is inevitable. A surgeon monitors a medical robot. A farmer supervises an agricultural swarm. A warehouse manager oversees a mixed fleet. A first responder needs to halt a malfunctioning robot. A child interacts with a home assistant. An AI system generates commands for a physical agent. At every level — from direct teleoperation to high-level supervision to emergency intervention — humans will interact with robots constantly, every day, at all times.

Today, every robot requires its own controller, its own interface, its own training. This fragmentation is unsustainable at scale. Just as the proliferation of automobiles required standardized controls so that any licensed driver could operate any vehicle safely, the proliferation of billions of robots demands a global common operator interface standard. Without it, robot operation remains a specialist skill, safety is inconsistent, fleet management is fragmented, and the potential of Physical AI is bottlenecked by interface incompatibility.

Billions of robots are coming — from nano to humanoid. Every human will deal with robots every day. A global common operator interface standard is not optional. RRCF is that standard.

2. Core Definition

RRCF is the Operator Interface Declaration Standard that defines how any robot is controlled — by human operators through a unified web UI, or by AI systems (VLA) through direct .rrcf profile loading at runtime — and also includes fleet management, collection and integration, safety guardrails, and choreography standards, complementing URDF, VDA 5050, and other existing standards to support most parts of Physical AI.

Key distinction: RRCF is an Operator Interface Declaration. VDA 5050 and Open-RMF are fleet mission assignment protocols. They operate at different layers and are complementary. RRCF sits above fleet protocols as the human/AI-facing operator layer.

3. The Five Pillars

1. REMOTE CONTROL STANDARD — One unified UI for any robot, any morphology
2. FLEET MANAGEMENT STANDARD — Complements VDA 5050 / Open-RMF (not competes)
3. COLLECTION & INTEGRATION STANDARD — Universal wire format for any external system
4. SAFETY STANDARD — E-stop, watchdog, geofence standardized across all robots
5. CHOREOGRAPHY STANDARD — Synchronized multi-robot missions (.rrcm files)

3.1 Pillar 1 — Remote Control Standard (Primary Purpose)

RRCF's primary purpose is a unified operator interface for any robot. A robot manufacturer publishes a .rrcf file. Any RRCF-compliant web controller reads it and renders the correct

interface — joysticks, mode buttons, skill actions, HUD strip, button legend, attachment tabs, custom controls — without per-robot code. Operator means human (web UI) or AI system (VLA via .rrcf profile).

3.2 Pillar 2 — Fleet Management Standard

RRCF complements VDA 5050 and Open-RMF — it does not compete with them. VDA 5050 defines how fleet software assigns missions to robots (backend, machine-to-machine). Open-RMF orchestrates multi-robot task allocation. RRCF defines how operators control and monitor robots (frontend, human-or-AI-to-machine). RRCF sits above these protocols: a RRCF-compliant robot declares its VDA 5050 endpoint within its .rrcf transport block, enabling unified operator control above the existing fleet protocol stack.

3.3 Pillar 3 — Collection & Integration Standard

Because RRCF defines the wire format for all robot commands and telemetry, the same format used for remote control becomes the universal integration format for any external system: data collection pipelines, VLA training datasets, digital twins, simulators, hospital management systems, ERP platforms, safety stacks, and cloud IoT services. One wire format connects robots to the entire software ecosystem — analogous to how USB's common standard enabled any device to connect to any computer.

3.4 Pillar 4 — Safety Standard

RRCF mandates E-stop in every .rrcf file. Every compliant robot must halt immediately on estop:true. RRCF also declares watchdog timeouts, geofence boundaries, and speed limits as operational guardrails. RRCF safety declarations complement NVIDIA Halos (IEC 61508 functional safety) and ISO 10218 — Halos ensures the robot does not fail internally; RRCF declares what the robot is permitted to do externally. Together they form a complete safety architecture for Physical AI.

3.5 Pillar 5 — Choreography Standard

RRCF defines .rrcm (RRCF Mission) files for synchronized multi-robot operations. Pre-computed per-robot trajectories trigger simultaneously via GPS clock sync. Covers drone light shows, robot dance performances, coordinated agricultural swarm operations, and emergency fleet coordination.

4. The Three-Layer Robotics Standard Stack

RRCF completes the three-layer robotics standard stack:

Layer	Standards	What it answers
Physical Layer	URDF, MJCF, SDF, Genesis, USD	What is this robot made of?
Fleet Layer	VDA 5050 v3.0, Open-RMF, AMRA-271	How are missions assigned to robots?
Operator Layer ← RRCF	RRCF — this standard	How does any operator control any robot?

The operator layer was empty before RRCF. Every robot vendor filled it with proprietary solutions. RRCF standardizes it for the first time.

5. Prior Art and Differentiation

Standard	What it covers	Relationship to RRCF	What RRCF adds
VDA 5050 v3.0 (2026)	Fleet mission assignment, JSON/MQTT, wheeled AGV/AMR only	RRCF complements — sits ABOVE in stack	Operator UI declaration, all 12 morphologies, skills, VLA, input modalities
Open-RMF	Multi-robot task allocation, traffic mgmt, ROS2	RRCF complements — different layer	Operator declaration file, non-ROS transports, all morphologies
AMRA-271:2025	Mobile robot comms + peripheral integration	RRCF complements	Operator UI, skill declaration, VLA integration, choreography
URDF / MJCF / SDF	Physical description — kinematics, geometry, physics	RRCF references, does not replace	Behavioral/operator interface layer above physics
NVIDIA Halos (2026)	Functional safety IEC 61508, hardware safety stack	RRCF complements — different layer	External behavioral contract, operator guardrails
USB HID / W3C Gamepad	Raw hardware input — axes and buttons	RRCF provides semantic layer above HID	Robot-specific axis mapping, mode/skill semantics per morphology

6. Morphology Category Taxonomy

RRCF-1.0 defines twelve canonical robot morphology categories plus an open custom category for future morphologies. A robot declares exactly one primary category. Additional categories may be declared as attachments.

ID	Category	Description	Examples
wheeled	Wheeled / Tracked	Differential, omni, tracked base	TurtleBot, Clearpath Husky, AMRs
legged	Legged (Quad/Hex)	Multi-legged, no arms	Unitree Go2, Spot, ANYmal
loco_manip	Loco-Manipulation	Legged base + arm(s)	Spot+Arm, Go2+Z1, ANYmal+ARM
wheeled_humanoid	Wheeled Humanoid	Wheeled base + full upper body	Sanctuary Phoenix Gen8
full_humanoid	Full Humanoid	Bipedal + full upper body	Unitree G1/H1, Figure 02, Optimus
manipulator	Fixed Manipulator	Stationary arm, no base	UR5/10, Piper AgileX, Franka

aerial	Aerial (Drone/VTOL)	Multirotor, fixed-wing, VTOL	DJI, ArduPilot, PX4 drones
marine_surface	Marine Surface (USV)	Unmanned surface vessel	WAM-V, ArduPilot Boat
marine_sub	Marine Underwater	ROV / AUV subsurface	BlueROV2, VideoRay ROV
industrial_vehicle	Industrial Vehicle	Excavator, dozer, forklift	CAT excavator, Komatsu dozer
agri_vehicle	Agricultural Vehicle	Sprayer, laser weeder, planter	John Deere autonomy, Naïo Oz
custom	Custom (Vendor-defined)	Any future morphology not in above	Medical, nano, space, soft robot

7. RRCF File Structure — .rrcf

A .rrcf file is an XML document. The file may be standalone or embedded within an existing robot description file (URDF, MJCF, etc.):

```
<?xml version="1.0" encoding="UTF-8"?>
<rrcf version="1.0" xmlns="https://rrcf.io/schema/1.0">
  <meta>                                <!-- Robot identity, vendor, physical refs -->
  <primary>                             <!-- Primary category + locomotion declaration -->
  <attachments>                         <!-- 0-N attachments: arm, gripper, sensor, tool -->
  <hud>                                <!-- HUD/LCD strip + context-sensitive button legend -->
  <skills>                             <!-- Standard + vendor-custom skill declarations -->
  <input_modalities>                   <!-- touch, HID gamepad, voice, gesture, XR -->
  <custom_controls>                   <!-- Up to 5 operator-defined controls -->
  <telemetry>                         <!-- Declared telemetry fields + warn thresholds -->
  <safety>                             <!-- E-stop, watchdog, geofence, speed limits -->
  <transport>                         <!-- MQTT, ROS2, WebSocket, VDA5050, REST -->
  <extensions/>                       <!-- Future RRCF-2.0 blocks, ignored by 1.0 -->
</rrcf>
```

7.1 Complete Example — Unitree Go2 + Z1 Arm

```
<rrcf version="1.0" xmlns="https://rrcf.io/schema/1.0">
  <meta>
    <name>Unitree Go2 + Z1 Arm</name>
    <vendor>Unitree Robotics</vendor>
    <model>Go2-Z1</model>
    <physical_ref format="urdf" path="go2_z1.urdf"/>
    <physical_ref format="mjcf" path="go2_z1.xml"/>
  </meta>
  <primary category="legged">
    <locomotion axes="vx vy wz" max_vx="1.5" max_vy="0.5" max_wz="2.0"/>
    <modes>stand trot bound crawl prone</modes>
    <body_pose pitch="true" roll="true" height="true"/>
  </primary>
  <attachments>
    <attach type="manipulator" id="arm_z1" mount="torso_top" dof="6">
```

```

    <modes>stow reach teach replay</modes>
    <ee_control cartesian="true"/>
  </attach>
  <attach type="gripper" id="grip_z1" mount="arm_z1/ee" fingers="2"/>
</attachments>
<hud>
  <field id="mode" position="1" label="MODE"/>
  <field id="battery" position="2" label="BATT"/>
  <field id="estop" position="3" label="STOP" alert="true"/>
  <button_legend>
    <mode name="trot">
      <key id="F1" label="SIT"/> <key id="F2" label="STAND"/>
      <key id="F3" label="BOUND"/>
    </mode>
    <mode name="reach">
      <key id="F1" label="GRIP"/> <key id="F2" label="RELEASE"/>
      <key id="F3" label="STOW"/>
    </mode>
  </button_legend>
</hud>
<skills>
  <skill id="sit" label="Sit" standard="true" cmd='{ "mode": "sit" }'/>
  <skill id="stand" label="Stand" standard="true" cmd='{ "mode": "stand" }'/>
  <!-- Vendor custom - VLA reads at runtime, no retraining needed -->
  <skill id="heel_stretch" label="Yoga Pose"
    description="Extends rear legs in yoga pose"
    cmd='{ "mode": "custom_01" }'/>
</skills>
<input_modalities>
  <modality type="touch_web" enabled="true"/>
  <modality type="hid_gamepad" enabled="true" standard="xinput">
    <axis hid="left_x" rrcf="lx"/>
    <axis hid="left_y" rrcf="ly" invert="true"/>
    <axis hid="right_x" rrcf="rx"/>
    <axis hid="right_y" rrcf="ry" invert="true"/>
    <button hid="A" action="skill:sit"/>
    <button hid="B" action="estop"/>
    <button hid="RB" action="mode_next"/>
  </modality>
  <modality type="voice" enabled="true" language="en-US">
    <phrase text="emergency stop" action="estop"/>
    <phrase text="go home" action="skill:home"/>
  </modality>
  <modality type="gesture_cam" enabled="true" backend="mediapipe">
    <gesture name="fist" action="estop"/>
    <gesture name="thumbs_up" action="skill:stand"/>
  </modality>
  <modality type="xr_controller" enabled="true" api="webxr">
    <hand side="left" maps_to="left_stick"/>
    <hand side="right" maps_to="right_stick"/>
    <trigger side="right" action="skill:grip"/>
  </modality>
</input_modalities>
<custom_controls max="5">
  <control type="toggle" id="led" label="LED Strip" default="off"/>

```

```

    <control type="slider" id="follow_dist" label="Follow Dist"
        min="0.5" max="3.0" unit="m"/>
    <control type="button" id="bark" label="Bark" action="momentary"/>
</custom_controls>
<telemetry>
    <field id="battery" unit="%" warn_below="20"/>
    <field id="temp" unit="C" warn_above="55"/>
    <field id="speed" unit="m/s"/>
    <field id="payload" unit="kg"/>
</telemetry>
<safety>
    <estop required="true" topic="/rrcf/go2/estop" qos="2"/>
    <watchdog timeout_ms="500" action="halt"/>
    <speed_limit max_vx="1.5" max_wz="2.0"/>
</safety>
<transport>
    <endpoint role="operator_cmd" protocol="rrcf_mqtt"
        topic="/rrcf/go2/cmd" broker="{MQTT_BROKER}"/>
    <endpoint role="telemetry" protocol="mqtt"
        topic="/rrcf/go2/state" frequency_hz="10"/>
    <endpoint role="ros2" protocol="ros2"
        topic="/cmd_vel" msg="geometry_msgs/Twist"/>
    <endpoint role="fleet" protocol="vda5050" version="3.0"
        topic="/uagv/v2/unitree/go2_001"/>
    <endpoint role="websocket" protocol="websocket" port="9090"/>
</transport>
<extensions/>
</rrcf>

```

8. Wire Format

All input modalities — touch, HID gamepad, voice, gesture, XR, VLA — produce the same RRCF wire format output. The robot receives one format regardless of how the operator is interacting:

```

{
  "rrcf": "1.0",          // Protocol version - always present
  "category": "legged",   // Morphology category
  "type": "cmd",          // Message type
  "lx": 0.75,             // Left stick X range [-1.0, 1.0]
  "ly": 0.00,             // Left stick Y range [-1.0, 1.0]
  "rx": -0.25,            // Right stick X range [-1.0, 1.0]
  "ry": 0.00,             // Right stick Y range [-1.0, 1.0]
  "speed": 0.60,          // Speed scalar range [0.0, 1.0]
  "mode": "trot",         // Active locomotion mode
  "skill": null,          // Active skill id or null
  "estop": false,         // E-stop - true = halt immediately
  "input": "touch_web",   // Which modality produced this command
  "twist": {              // ROS2 geometry_msgs/Twist - always present
    "linear": {"x":0.75, "y":0.00, "z":0.00},
    "angular": {"x":0.00, "y":0.00, "z":-0.25}
  },
  "ts": 1719000000000     // Unix timestamp milliseconds
}

```

9. VLA Integration — RAG for Robots

RRCF enables Vision-Language-Action models to control any compliant robot without fine-tuning, via Retrieval-Augmented Generation:

- At session start, the VLA loads the robot's .rrcf file as structured context.
- Declared skills extend the VLA's action vocabulary at runtime — no retraining.
- Vendor-custom skills (heel_stretch, seed_row_align) become available to the VLA instantly because the skill declaration IS the context.
- The VLA generates RRCF wire format commands bounded by declared safety limits.
- This solves the combinatorial explosion: thousands of robots x hundreds of skills = millions of skills impossible to train. RRCF makes each robot's skills available at runtime.

RAG for Robots: instead of retrieving documents, the VLA retrieves the robot's behavioral profile. Load the .rrcf file, know the robot, control it.

10. Input Modalities

RRCF-1.0 defines five input modality types. All produce identical wire format:

Modality	Technology	RRCF-1.0	Notes
touch_web	Browser touch / mouse	Required — baseline	On-screen joysticks rendered from .rrcf
hid_gamepad	W3C Gamepad API / USB HID	Optional	Xbox/PS5/any HID. Mapping declared in .rrcf
voice	Web Speech API	Optional	Skill names become voice commands automatically
gesture_cam	MediaPipe / camera	Optional	Hand gestures mapped to RRCF actions
xr_controller	WebXR Device API	Optional	XR hand pose maps to joystick axes + skills

11. Conformance Requirements

11.1 RRCF-Compliant Robot MUST:

- Include a valid <rrcf version="1.0"> block in robot description or standalone .rrcf file.
- Accept RRCF wire format JSON on at least one declared transport endpoint.
- Implement E-stop: upon estop:true, halt ALL motion immediately, no exceptions.
- Implement watchdog: halt if no valid RRCF command within declared timeout.
- Publish telemetry on declared fields at minimum 1 Hz.

11.2 RRCF-Compliant Controller MUST:

- Parse .rrcf file without requiring any other description format.
- Render core panel (joysticks, E-stop, speed, telemetry) for ALL categories.
- Render profile panel from declared category, modes, skills, attachments.
- Render HUD strip from <hud> fields and update button legend per active mode.
- Support touch_web input modality as baseline.
- Include estop field in EVERY transmitted command message.
- Normalize all input axes to range [-1.0, 1.0] in wire format output.
- Include rrcf version string and Twist sub-object in every command message.

12. Versioning and Governance

RRCF uses semantic versioning. RRCF-1.0 is the initial standard. Certification authority is held by Indo-Mars Technologies / MarsGeo Platform. Category proposals follow: proposal → 60-day review → draft → 3 independent implementations → ratification → publication.

Version	Planned additions	New categories
RRCF-1.0	12 categories, all 5 pillars, 5 input modalities	wheeled, legged, loco_manip, wheeled_humanoid, full_humanoid, manipulator, aerial, marine_surface, marine_sub, industrial_vehicle, agri_vehicle, custom
RRCF-1.1	Voice modality refinements, gesture spec, XR training	No new base categories — attachment type additions
RRCF-2.0	World context block, benchmarking block, RL reward hints	medical_endoscopic, surgical, micro_robot, exoskeleton, soft_robot, swarm_node
RRCF-3.0	Neural interface, zero-G thruster primitive	nano_robot, space_zero_g, neural_interface

13. Security Considerations

- Transport encryption: MQTT MUST use TLS (MQTTS port 8883). WebSocket MUST use WSS in production.
- Authentication: Endpoints SHOULD require auth. Wire format contains no auth fields — handled at transport layer.
- E-stop hardware independence: Robots MUST implement hardware watchdog independent of network.
- VLA guardrails: Controllers and VLA integrations MUST reject commands exceeding declared safety limits.
- Command rate limiting: Robots SHOULD implement rate limiting against denial-of-service.

14. References

- [RFC2119] Key words for RFCs, Bradner, 1997.
- [VDA5050] VDA 5050 v3.0, VDA/VDMA, March 2026.

- [OPEN-RMF] Open Robotics Middleware Framework, OSRA, 2024.
 - [AMRA-271] AMRA-271:2025, Autonomous Mobile Robot Alliance, 2025.
 - [ANSI-A3] ANSI/A3 R15.06-2025, A3/ANSI, October 2025.
 - [HALOS] NVIDIA Halos Robotics Safety System, NVIDIA, June 2026.
 - [W3C-PAD] Gamepad API, W3C, 2024.
 - [WEBXR] WebXR Device API, W3C, 2024.
 - [RRCF-REF] RRCF Reference Implementation, Indo-Mars Technologies, July 8, 2026.
-

Author

[YOUR FULL LEGAL NAME] | Indo-Mars Technologies / MarsGeo Platform

[YOUR ADDRESS] | [EMAIL] | indo-mars.com | rrcf.io (pending)

Copyright (c) 2026 [YOUR FULL LEGAL NAME]. Apache License 2.0.